

W2HP: A Hierarchical Multi-Agent Reinforcement Learning Framework for Multi-Domain Service Function Chain Deployment

Peiying Zhang , *Member, IEEE*, Shengpeng Chen, Chunming Rong , *Senior Member, IEEE*, Ching-Hsien Hsu , *Senior Member, IEEE*, Neeraj Kumar , *Senior Member, IEEE* and Mohsen Guizani , *Fellow, IEEE*

Abstract—As network function virtualization (NFV) expands across administrative boundaries, multi-domain service function chain placement (MD-SFCP) faces a critical dilemma: achieving global optimization requires sharing sensitive topology information, while domain autonomy mandates strict privacy protection. Existing solutions typically fall into two extremes: constructing a quasi-global view via heavy state exchange, which compromises privacy and scalability, or relying on independent multi-agent learning that optimizes intra-domain placement (*Where*) but neglects the critical domain-selection decision (*Which*). Consequently, these methods fail to explicitly evaluate domain-level capabilities prior to node-level assignment, leading to load imbalance and resource inefficiency. To bridge this gap, we propose W2HP, a Which-and-Where Hierarchical Placement framework. W2HP employs an information-isolated centralized-distributed architecture, where each domain retains its fine-grained local state and exposes only an encoded domain representation to the high-level central agent for domain selection, thereby avoiding direct exchange of raw intra-domain resource states. Inside domains, low-level agents execute fine-grained node placement. We design a dual-encoder network to process hierarchical states and utilize Multi-Agent Proximal Policy Optimization (MAPPO) with a centralized critic to stabilize cooperative training. Extensive experiments on three network scales demonstrate that W2HP significantly outperforms state-of-the-art baselines, improving the acceptance ratio by up to 8.3% in large-scale scenarios and increasing the revenue-to-cost ratio by 12.26%, proving its efficacy in balancing cross-domain coordination with local autonomy.

This work is partially supported by the National Natural Science Foundation of China under Grant 62471493, partially supported by the Natural Science Foundation of Shandong Province under Grant ZR2023LZH017 and ZR2024MF066. (*Corresponding author: Peiying Zhang*).

Peiying Zhang and Shengpeng Chen are with the Qingdao Institute of Software, College of Computer Science and Technology, China University of Petroleum (East China), Qingdao 266580, China, and also with the Shandong Key Laboratory of Intelligent Oil & Gas Industrial Software, Qingdao 266580, China (e-mail: zhangpeiying@upc.edu.cn; z24070039@s.upc.edu.cn).

Chunming Rong is with the Department of Electronic Engineering and Computer Science, University of Stavanger, 4036 Stavanger, Norway (e-mail: chunming.rong@uis.no).

Ching-Hsien Hsu is with the Department of Computer Science and Information Engineering, Asia University, Taichung 41354, Taiwan, and also with the Department of Medical Research, China Medical University Hospital, China Medical University, Taichung 404, Taiwan (e-mail: robertchh@gmail.com).

Neeraj Kumar is with the Department of Computer Science and Engineering, Thapar Institute of Engineering and Technology, Patiala 147004, India, and also with the Department of Networks and Communications, College of Computer Science and Information Technology, Imam Abdulrahman Bin Faisal University, Dammam 31441, Saudi Arabia (e-mail: neeraj.kumar@thapar.edu).

Mohsen Guizani is with the Machine Learning Department, Mohamed Bin Zayed University of Artificial Intelligence (MBZUAI), Abu Dhabi 999041, UAE (e-mail: mguizani@ieee.org).

Index Terms—Service Function Chain, Multi-domain Network, Hierarchical Reinforcement Learning, Privacy-Constrained, Resource Allocation

I. INTRODUCTION

A. Background

WITH the emergence of sixth-generation (6G) networks and the Industrial Internet of Things (IIoT), emerging applications require stringent, deterministic, and differentiated quality-of-service (QoS) guarantees [1], [2]. To satisfy these requirements, service delivery typically spans multiple administrative domains (e.g., distinct ISPs or cloud providers), thereby necessitating the orchestration of Service Function Chains (SFCs) across heterogeneous infrastructures. Although Network Function Virtualization (NFV) and Software-Defined Networking (SDN) offer the architectural flexibility to support such services [3], the transition from single-domain to multi-domain environments introduces significant challenges regarding coordination and privacy.

In typical Multi-Domain SFC Placement (MD-SFCP) scenarios, an SFC request entails the sequential traversal of Virtual Network Functions (VNFs) distributed across distinct domains. In contrast to single-domain settings where a global controller maintains comprehensive visibility, multi-domain environments are characterized by information isolation. Motivated by business competition and security policies, administrative domains generally avoid disclosing internal network topologies or fine-grained resource states [4], [5]. Moreover, the scale of modern networks renders centralized global optimization impractical due to excessive signaling overhead and computational latency [6]. Consequently, the primary challenge in MD-SFCP is to jointly improve request admission, embedding cost efficiency, and end-to-end SFC delay, while respecting the local autonomy and information-isolation requirements of individual domains. Here, local autonomy means that each administrative domain retains control over its internal resource state and local placement decisions, while exposing only the abstract information required for cross-domain coordination.

B. Related Work

Prior studies on SFC deployment can be broadly divided into single-domain and multi-domain settings.

TABLE I: Comparison of Representative Works on Multi-Domain SFC Deployment.

Work	Problem Type	Method	Control Architecture		Privacy	Solution Architecture	
			Distributed	Centralized		Which	Where
Lin et al. [7]	Placement	Heuristic	✓		Protected		✓
Sun et al. [8]	Placement	Heuristic	✓	✓	Protected		✓
Zhao et al. [9]	Routing	MARL	✓		Open		✓
Wang et al. [10]	Placement	MARL	✓		Open		✓
Zhang et al. [11]	Placement	Heuristic		✓	Open		✓
Zhang et al. [12]	Placement	RL	✓		Open		✓
Zhang et al. [13]	Placement	FL	✓		Protected		✓
Thakur et al. [14]	Migration	Heuristic		✓	Open		✓
Toumi et al. [15]	Placement	Heuristic	✓		Open		✓
Toumi et al. [16]	Placement	MARL	✓		Open	✓	✓
Cai et al. [4]	Placement	RL	✓	✓	Protected		✓
Zhang et al. [17]	Migration	Heuristic		✓	Open		✓
Ours (W2HP)	Placement	MARL	✓	✓	Protected	✓	✓

1) *Single-Domain SFC Deployment*: Research on single-domain SFC deployment mainly focuses on three aspects: optimization-based placement and routing, resource- and state-aware deployment, and distributed scheduling. Early studies examined VNF placement [18] and chain routing [19], often formulating the problem as integer linear programming (ILP) or mixed-integer linear programming (MILP) to optimize latency or deployment cost [2], [20]. Since these exact formulations do not scale well, many later approaches turned to heuristics based on link ordering, node filtering, and local search [21]–[23]. More recent work has shifted from single-objective placement to resource- and state-aware deployment. This line includes deep reinforcement learning (DRL) for objective-aware placement [24], DRL-based caching to reduce VNF cold-start delay in edge environments [25], reliability-aware deployment for stateful VNFs in dynamic settings [26], and backup optimization in edge networks [27]. Another direction considers distributed deployment and parallel scheduling, including resource-aware matching with improved deep Q-network (DQN) policies [28], online placement over geo-distributed data centers without future traffic knowledge [29], load-aware scheduling to improve the acceptance ratio [30], and delay balancing across parallel modules to reduce packet accumulation [31].

2) *Multi-Domain SFC Deployment*: Multi-domain SFC deployment is more challenging because it must cope with topology opacity, policy heterogeneity, and cross-domain state migration [32]. Existing studies can be roughly grouped into decomposition-based embedding, learning-based coordination, and privacy- or architecture-aware designs. On the optimization side, decomposition methods such as column generation divide the global problem into per-domain subproblems without exposing intra-domain details [7], while abstract-backbone and partition-and-feedback schemes improve mapping success and bandwidth efficiency [8]. Sun et al. [8] employ Full-Mesh Aggregation (FMA) to construct an abstract multi-domain topology while hiding the detailed intra-domain topology. In the resulting representation, border nodes are retained and intra-domain connectivity between border-node pairs is represented by aggregated logical links. Based on this abstract topology, their method further performs SFC partitioning and

sub-chain deployment across domains. Other work extends cross-domain embedding to more realistic settings by jointly considering software availability and migration [17], inter-domain flow migration and load redistribution [14], and phased mapping with path splitting for delay reduction [11]. These studies also suggest that classical virtual network embedding (VNE) techniques remain useful for cross-domain optimization. Learning-based coordination has also received increasing attention, including cooperative multi-agent reinforcement learning (MARL) for cross-domain routing [9], digital twins combined with multi-agent deep deterministic policy gradient (MADDPG) under centralized training with decentralized execution (CTDE) [10], federated DRL for privacy-preserving scheduling without sharing raw topology data [13], and graph neural network (GNN)-enhanced DRL for dynamic partitioning and policy learning [12]. Privacy preservation and interoperability have also been studied through intent abstraction with DQN-based abstract search [4] and hierarchical orchestration across heterogeneous encapsulation and forwarding technologies [15].

C. Motivations

To understand why domain selection becomes an explicit decision in MD-SFCP, we distinguish the multi-domain setting from fully observable SFC placement. As illustrated in Fig. 1(a), when the complete physical network is visible to a controller, each VNF can be assigned to a node in the global candidate set. Since selecting a physical node also determines its administrative domain, a separate domain-selection action is unnecessary. This equivalence no longer holds when administrative domains do not expose their fine-grained topology and resource states. As illustrated in Fig. 1(b), the central controller cannot directly select a physical node from the union of all intra-domain node sets; instead, a hosting domain must be determined before the corresponding local controller can perform node placement. An inappropriate domain decision generally cannot be corrected by a subsequent intra-domain policy. If the selected domain does not contain a feasible node for the current VNF, the local policy can only reject the placement even when feasible nodes remain available in another domain.

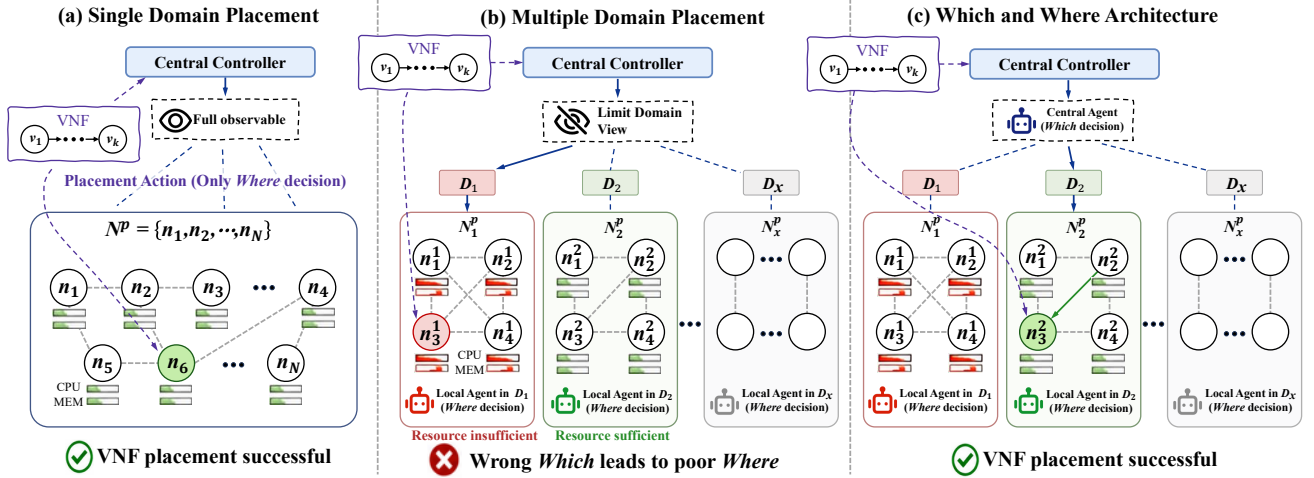


Fig. 1: Why an explicit Which decision is required in privacy-constrained multi-domain SFC placement. (a) In a fully observable single-domain network, node selection determines the VNF location. (b) Under multi-domain information isolation, a local placement policy can select nodes only within the VNF’s assigned domain; an inappropriate domain assignment may therefore cause avoidable rejection even when another domain remains feasible. (c) W2HP selects a domain according to abstract domain representations and then performs fine-grained node placement using private intra-domain states.

Therefore, the relevant challenge is not the hierarchical decomposition itself, which also appears in prior multi-domain orchestration schemes [16], but how to coordinate domain selection and intra-domain placement under an information boundary between administrative domains. W2HP addresses this problem by allowing the high-level policy to compare abstract domain representations and subsequently invoking a fine-grained placement policy only within the selected domain, as shown in Fig. 1(c). This failure mechanism is supported by the ablation results in Section IV-C. At an arrival rate of 0.6, replacing the learned **Which** policy with **random domain selection** decreases the stabilized acceptance ratio by 9.07-66.91 percentage points (pp) and long-term revenue-to-cost ratio by 343-2171 across the four evaluated topologies. Compared with the **resource-greedy domain-selection heuristic**, the learned policy improves the acceptance ratio by 0.45-10.47 pp and LT_R2C by 53-291. These results indicate that the domain-selection strategy directly affects the feasible action space of the subsequent Where policy, although the additional benefit of learned domain selection over a well-designed heuristic varies with the network topology.

D. Contributions

To mitigate the aforementioned limitations, we propose the **Which-and-Where Hierarchical Placement (W2HP)** framework. This framework decomposes the MD-SFCP process into two hierarchical sub-problems: global domain selection and local node placement. Specifically, we design a collaborative centralized-distributed architecture wherein domain agents (operating at the lower level) manage local physical networks and transmit only abstract *resource quality scores* to a central controller (operating at the higher level). This mechanism enables the central agent to execute informed domain-selection decisions without requiring access to sensitive internal data. By leveraging a dual-encoder design in conjunction with

Multi-Agent Proximal Policy Optimization (MAPPO), W2HP effectively optimizes the trade-off between global load distribution and local resource efficiency. The primary contributions of this paper are summarized as follows:

- **Decomposed Privacy-Constrained Framework:** We introduce a hierarchical decision-making framework that resolves the conflict between cross-domain coordination and information isolation. Unlike existing distributed approaches that neglect domain-level capabilities, our method explicitly evaluates domain suitability via abstract representations, enabling optimized inter-domain orchestration without exposing sensitive intra-domain details.
- **Heterogeneous Policy Network:** We propose a specialized dual-encoder policy network: a high-level encoder for abstract domain scoring and a low-level encoder for fine-grained node selection. Coupled with a centralized-critic MAPPO training strategy, this architecture effectively mitigates the non-stationarity inherent in multi-scale state spaces.
- **Framework Feasibility:** Based on the Cooperative Routing Mechanism, we provide a theoretical proof of its path optimality under topology abstraction. Empirical results validate that W2HP achieves superior scalability and resource efficiency compared to both heuristic and learning-based baselines, improving the request acceptance ratio by up to 8.3% in large-scale scenarios and increasing the long-term Revenue-to-Cost ratio by 12.26%.

The remainder of this paper is organized as follows. Section II elaborates on the system model and problem formulation. Section III introduces the W2HP algorithm and the corresponding training methodology. Section IV presents the performance evaluation, and Section V concludes the paper.

TABLE II: Summary of major notations.

Symbol	Definition	Symbol	Definition
G	Multi-domain physical network	g_d^p	Physical network of domain d
N_d^p, L_d^p	Node and link sets of domain d	L^{global}	Set of inter-domain physical links
N^p	Set of all physical nodes across domains	$n_{d,i}^p, l_{d,i}^p$	i -th physical node and link in domain d
$C_{n_{d,i}^p}, M_{n_{d,i}^p}$	CPU and memory capacities of node $n_{d,i}^p$	$BW_{l_{d,i}^p}$	Bandwidth capacity of physical link $l_{d,i}^p$
$g_{t,k}^v$	k -th SFC request arriving at time slot t	N^v, L^v	Sets of VNFs and virtual links of an SFC request
$C_{n_i^v}, M_{n_i^v}$	CPU and memory demands of VNF n_i^v	$BW_{l_i^v}$	Bandwidth demand of virtual link l_i^v
$\hat{r}_n$	VNF-to-physical-node mapping function	$\hat{r}_l$	Virtual-link-to-physical-path mapping function
$P^p(u, v)$	Set of physical paths between nodes u and v	$\Omega(g^v)$	Binary indicator of embedding of request g^v
s_d^p	Fine-grained state of domain d	s^v	State of the current SFC request
S^H	High-level state used for domain selection	S^L	Low-level state used for node placement
A^H	High-level action, i.e., selected domain	A^L	Low-level action, i.e., selected physical node
π^H, π^L	High- and low-level policies	R^H, R^L	Rewards of high- and low-level actions
$U_{C,d}, U_{M,d}, U_{BW,d}$	Resource utilization ratios of domain d	$\mathbf{m}_d$	Resource-feasibility mask for nodes in domain d
h_d^p, h^v	Features of the physical domain and SFC	$\bar{Z}^H, \bar{Z}^L$	Logits generated for high- and low-level policies
P_d^H	Probability of selecting domain d	$P_{n_{d,i}^p}^L$	Probability of selecting physical node $n_{d,i}^p$
G^{abs}	Abstract inter-domain graph used by CRM	P^{abs}	Shortest path computed on the abstract graph
d_s, d_t	Source and destination domains	P_i	Physical path assigned to virtual link l_i^v
θ^H, θ^L, d	Parameters of the high- and low-level policies	ϕ	Parameters of the centralized critic
V_ϕ	Centralized state-value function	B	Buffer containing completed SFC trajectories
N_{map}	Set of mapped physical nodes	L_{map}	Set of mapped physical paths
s_t, a_t, R_t	State, action, and reward at training step t	γ	Discount factor
λ	GAE decay parameter	ϵ	PPO clipping parameter
$\hat{A}_t$	Generalized advantage estimate at step t	δ_t	Temporal-difference residual at step t
$\hat{R}_t$	Target return used to train the critic	c_1, c_2, c_3	Weights of policy, value, and entropy terms
K_{epochs}	Number of PPO update epochs	η	Learning rate

II. SYSTEM MODEL

For ease of reference, the major notations used throughout this paper are summarized in Table II.

A. Multi-Domain Physical Network

The physical network is represented as a set of physical nodes and links with associated resource weights, formally defined as $G = \{g_1^p, g_2^p, \dots, g_d^p \mid L^{\text{global}}\}$. Here, g_d^p denotes the physical network of domain d , and L^{global} represents the set of inter-domain links. Specifically, the network of domain d is defined as a weighted undirected graph $g_d^p = \{N_d^p, L_d^p, AN_d^p, AL_d^p\}$, where N_d^p and L_d^p denote the sets of physical nodes and intra-domain physical links, respectively, alongside their corresponding attributes. The resource attributes of the i -th physical node in domain d are denoted by $n_{d,i}^p$, which encompasses computing and storage capacities, represented by $C_{n_{d,i}^p} \in AN_d^p$ and $M_{n_{d,i}^p} \in AN_d^p$, respectively. Similarly, the bandwidth of the physical link $l_{d,i}^p$ is denoted as $BW_{l_{d,i}^p} \in AL_d^p$.

B. Service Function Chain Request

A Service Function Chain Request (SFCR) orchestrates Virtual Network Functions (VNFs) in a specified sequence, ensuring that traffic traverses each VNF node in the designated order. An SFCR can be modeled as a weighted directed acyclic graph (DAG) $g^v = \{N^v, L^v, AN^v, AL^v\}$. The computing resources required by the virtual request are represented as

$AN^v = \{C_{n_i^v}, M_{n_i^v}\}$ for $i \in \{1, 2, \dots, |N^v|\}$, and the bandwidth of each directed virtual link is given by $AL^v = \{BW_{l_i^v}\}$ for $i \in \{1, 2, \dots, |L^v|\}$.

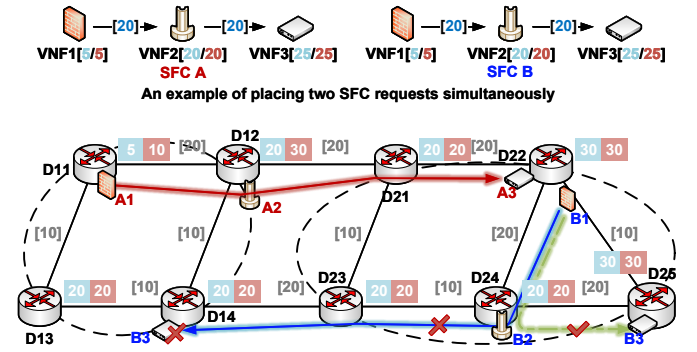


Fig. 2: Illustrative multi-domain embedding of two SFC requests. The dashed boundaries delimit network domains, and the router-shaped symbols denote physical nodes, where D_{ij} represents the j -th physical node in domain i . Values in $VNF_k[./.]$ denote the CPU and memory demands of the VNF. Values in square brackets on virtual and physical links denote required and available bandwidth, respectively. Labels A1–A3 and B1–B3 indicate the physical placements of the three VNFs in SFC A and SFC B, respectively.

C. Problem Definition

As illustrated in Fig. 2, the Multi-Domain SFC Placement (MD-SFCP) problem entails determining: (1) the location in

the physical network where each virtual node is hosted, and (2) the route along a physical path for each virtual link, while satisfying the computing, storage, and bandwidth constraints. Accordingly, we decompose the MD-SFCP problem into two subproblems: the Node Placement Problem (NPP) and the Link Mapping Problem (LMP). An embedding is characterized by a pair of mapping functions $(\hat{r}_n, \hat{r}_l)$, where $\hat{r}_n : N^v \rightarrow N^p$ denotes the node placement function and $\hat{r}_l : L^v \rightarrow \mathcal{P}^p$ denotes the link mapping function, with $\mathcal{P}^p$ representing the set of physical paths in G . The MD-SFCP problem is formally defined as:

$$\mathbf{P} : \quad \text{find } (\hat{r}_n, \hat{r}_l), \quad (1)$$

such that $\hat{r}_n$ is feasible for **P1** (NPP, Definition 1) and $\hat{r}_l$ is feasible for **P2** (LMP, Definition 2); that is, all node and link resource constraints are satisfied.

Definition 1 (P1. Node Placement Problem). *Given an SFC request with virtual nodes N^v , for each $n_i^v \in N^v$, the notation $\hat{r}_n(n_i^v) = n_{d,j}^p$ indicates that the virtual node n_i^v is hosted on the physical node $n_{d,j}^p$. The mapping $\hat{r}_n$ constitutes a feasible solution to the NPP if it satisfies the following conditions:*

- **Injectivity:** For all $n_i^v, n_k^v \in N^v$ with $n_i^v \neq n_k^v$, it holds that $\hat{r}_n(n_i^v) \neq \hat{r}_n(n_k^v)$.
- **Full assignment:** $|\text{Im}(\hat{r}_n)| = |N^v|$, where $\text{Im}(\hat{r}_n) = \{\hat{r}_n(n_i^v) \mid n_i^v \in N^v\}$; this implies that each $n_i^v \in N^v$ is assigned to exactly one physical node.
- **Resource feasibility:** For all $n_i^v \in N^v$, given $\hat{r}_n(n_i^v) = n_{d,j}^p$, the residual resources satisfy $\widehat{C}_{n_{d,j}^p} \geq C_{n_i^v}$ and $\widehat{M}_{n_{d,j}^p} \geq M_{n_i^v}$.

Definition 2 (P2. Link Mapping Problem). *Given an SFC request with virtual links L^v , for each $l_i^v \in L^v$, the notation $\hat{r}_l(l_i^v) = P^p$ indicates that the virtual link l_i^v is mapped to a physical path P^p . The mapping $\hat{r}_l$ constitutes a feasible solution to the LMP if it satisfies the following conditions:*

- **Path mapping:** For all $l_i^v \in L^v$, $\hat{r}_l(l_i^v)$ comprises a valid physical path consisting of one or more physical links.
- **Bandwidth feasibility:** For all $l_i^v \in L^v$ and for every physical link $l_{d,i}^p \in \hat{r}_l(l_i^v)$, the residual bandwidth satisfies $\widehat{BW}_{l_{d,i}^p} \geq BW_{l_i^v}$.

Furthermore, for each $l_i^v \in L^v$ with endpoints n_i^v and n_k^v , the LMP solution $\hat{r}_l(l_i^v)$ must form a simple physical path connecting the mapped nodes $\hat{r}_n(n_i^v)$ and $\hat{r}_n(n_k^v)$, i.e., $\hat{r}_l(l_i^v) \in \mathcal{P}^p(\hat{r}_n(n_i^v), \hat{r}_n(n_k^v))$.

The identification of a feasible solution $(\hat{r}_n, \hat{r}_l)$ implies that, for a given SFCR, we have obtained: (i) a node mapping $\hat{r}_n$ that places each VNF onto a physical node while satisfying resource requirements, and (ii) a corresponding link mapping $\hat{r}_l$ that establishes connectivity among the placed VNFs, thereby embedding the SFCR onto the physical network. We employ a binary variable $\Omega(g^v)$ to indicate whether the SFCR g^v is successfully embedded. If problem **P** admits a feasible solution for the instance, we set $\Omega(g^v) = 1$; otherwise, $\Omega(g^v) = 0$.

D. Performance Metrics

In this study, the network state is discretized into evenly spaced time slots of length Δt . Each arriving SFCR is indexed by its arrival slot; for instance, $g_{t,k}^v$ denotes the k -th request arriving at time t . At the system level, our objective is to maximize (i) the request acceptance ratio (Acc) within the multi-domain network and (ii) the overall embedding profit. Specifically, the acceptance ratio is defined as the fraction of SFCR requests that are successfully embedded relative to the total number of arriving requests. Over a time interval $[t_s, t_e]$, Acc is computed as:

$$Acc = \frac{\sum_{t=t_s}^{t_e} \sum_{k=1}^K \Omega(g_{t,k}^v)}{\sum_{t=t_s}^{t_e} \sum_{k=1}^K \left\lceil \frac{\Omega(g_{t,k}^v) + 1}{2} \right\rceil}. \quad (2)$$

Furthermore, we utilize the Revenue-to-Cost Ratio ($R2C$) to quantify the embedding utility of a single SFCR. The revenue of an SFCR corresponds to the total amount of resources requested, whereas the cost corresponds to the actual resource expenditure incurred during embedding. This metric is defined as:

$$R2C = \frac{\sum_{n_i^v \in N^v} (C_{n_i^v} + M_{n_i^v}) + \sum_{l_i^v \in L^v} BW_{l_i^v}}{\left[\sum_{n_i^v \in N^v, n_{d,k}^p = \hat{r}_n(n_i^v)} (C_{n_{d,k}^p} u_C + M_{n_{d,k}^p} u_M) \right] + \sum_{l_i^v \in L^v, P^p = \hat{r}_l(l_i^v)} \left(\sum_{l_{d,k}^p \in P^p} BW_{l_{d,k}^p} \right) u_{BW}}, \quad (3)$$

where u_C , u_M , and u_{BW} denote the unit prices of computing, memory, and bandwidth resources, respectively. Additionally, the Long-Term $R2C$ is defined as:

$$LT_R2C = \sum_{t=0}^{T \rightarrow \infty} \sum_{k=1}^K R2C(g_{t,k}^v). \quad (4)$$

III. ALGORITHM DESIGN

A. Central-Distributed Architecture

We adopt a two-level hierarchical control architecture, wherein a central controller orchestrates M domain-level SDN controllers, each responsible for the control plane of a single network domain. The central controller receives aggregated representations of local observations from the corresponding domain controllers to facilitate global decision-making. To derive informative domain representations, we employ neural networks to extract resource and topology features from the physical network; consequently, we formulate this process as a hierarchical multi-agent reinforcement learning problem. Within this framework, we propose a multi-domain SFC deployment algorithm based on MAPPO. Specifically, the central controller executes a high-level domain selection policy, which accepts domain-level vectors produced by the high-level encoders of all domains as input and identifies the target domain for the current VNF. Subsequently, the controller of the selected domain executes a low-level node placement policy; this policy utilizes a fine-grained state representation generated by the low-level encoder of the domain to determine the

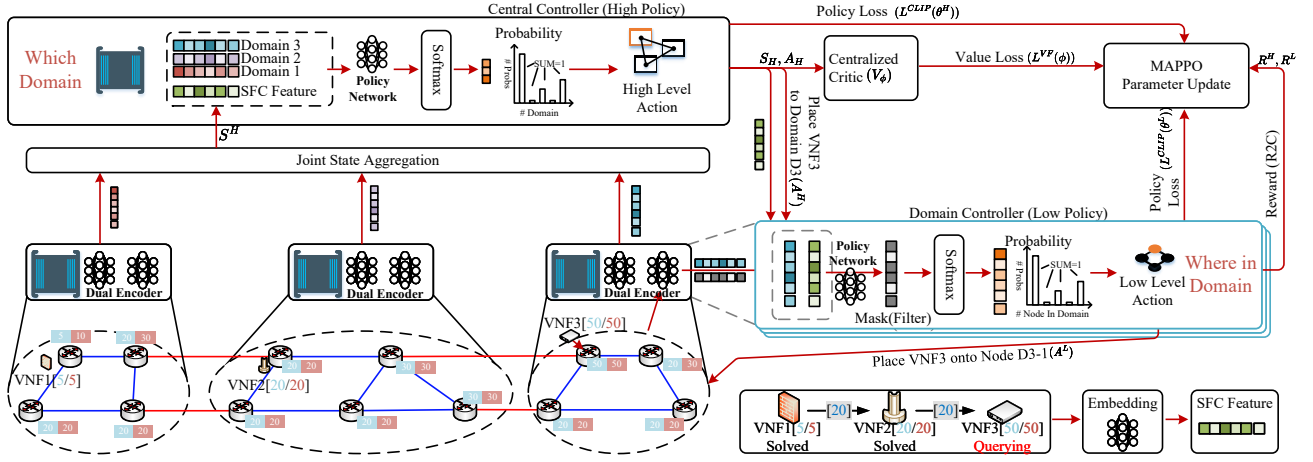


Fig. 3: Hierarchical Multi-Agent Architecture for Multi-Domain SFC Deployment Based on MAPPO.

Algorithm 1 CRM-based Link Mapping

- 1: **Input:** Abstract inter-domain graph G_{abs} , node mapping $\hat{r}_n$, virtual link set L^v .
- 2: **Output:** Link mapping $\hat{r}_l$.
- 3: $\hat{r}_l \leftarrow \emptyset$
- 4: **for** each virtual link $l_i^v = (n_{i,\text{src}}^v, n_{i,\text{dst}}^v) \in L^v$ **do**
- 5: $s \leftarrow \hat{r}_n(n_{i,\text{src}}^v)$, $t \leftarrow \hat{r}_n(n_{i,\text{dst}}^v)$
- 6: Determine the source domain d_s and destination domain d_t
- 7: **if** $d_s = d_t$ **then**
- 8: Compute local shortest path P_i from s to t
- 9: **else**
- 10: Step 1. Query C_{d_s} and C_{d_t} for access costs between $\{s, t\}$ and border nodes
- 11: Step 2. Add the temporary access links into G_{abs}
- 12: Step 3. Compute abstract shortest path P_{abs} from s to t
- 13: Step 4. Restore each abstract intra-domain segment in P_{abs} to its physical path
- 14: Step 5. Concatenate restored intra-domain paths and inter-domain links to obtain P_i
- 15: **end if**
- 16: $\hat{r}_l(l_i^v) \leftarrow P_i$
- 17: **end for**
- 18: **return** $\hat{r}_l$

target physical node. Given that the high-level representation must encapsulate abstract features comparable across domains, whereas the low-level representation requires sensitivity to intra-domain resource variations, we employ distinct encoders for the two policies. This design effectively mitigates gradient interference and enhances convergence stability. The structure of the proposed framework is illustrated in Fig. 3.

B. Cooperative Routing Mechanism for Link Mapping

After the hierarchical policy determines the node mapping $\hat{r}_n$, the remaining task is deriving the link mapping $\hat{r}_l$. For a virtual link $l_i^v = (n_{i,\text{src}}^v, n_{i,\text{dst}}^v)$, let $s = \hat{r}_n(n_{i,\text{src}}^v)$ and

$t = \hat{r}_n(n_{i,\text{dst}}^v)$. With $\hat{r}_n$ fixed, the routing subproblem becomes a point-to-point path query between s and t . However, the complete physical topology is unavailable to the central controller. Following Full-Mesh Aggregation (FMA) in [8], each domain d exposes an abstract topology containing its border nodes $\mathcal{B}_d$. For each pair $b_i, b_j \in \mathcal{B}_d$, an aggregated logical link represents their intra-domain shortest-path cost, while the detailed internal nodes and links remain local to the domain controller. Together with physical inter-domain links, these aggregated links form the abstract graph G_{abs} visible to the central controller. FMA provides the topology representation required for cross-domain coordination, but it does not alone specify the complete link-mapping procedure for the arbitrary physical endpoints produced by W2HP. In particular, the mapped nodes s and t are generally intra-domain rather than border nodes and thus are not directly represented in G_{abs} . Moreover, an abstract path must be translated back into executable intra-domain physical segments. Based on this abstraction, CRM performs a query-specific routing procedure. The source and destination domain controllers report the access costs from s and t to their border nodes. These temporary access links are added to G_{abs} , then the central controller computes an abstract shortest path. The involved domain controllers restore each aggregated intra-domain link to its physical path, and these segments are concatenated with the inter-domain links to obtain the final physical path P_i . Algorithm 1 summarizes this procedure.

For a fixed node mapping $\hat{r}_n$, CRM preserves the shortest-path objective provided that every aggregated intra-domain link and every query-specific endpoint-access link carries the exact shortest-path cost computed on the same physical graph used for the routing query. Under this condition, any physical path has an equal-cost abstract representation, and each abstract path selected by CRM can be restored to an equal-cost physical path. Consequently, the minimum path cost obtained by CRM equals that from shortest-path computation on the complete physical topology for this routing subproblem. When bandwidth feasibility is considered, the above condition is applied to the current residual network after excluding physical links that cannot satisfy the bandwidth demand of the

virtual link. Thus, the equivalence concerns the shortest-path link-mapping subproblem for a fixed VNF placement, rather than the global optimality of MD-SFCP. The formal proof is provided in Appendix A of the Supplementary File.

C. Hierarchical Multi-Domain Placement Policy

The difficulty of formulating MD-SFCP as a hierarchical MARL problem lies in the dependency between the two decision levels. The high-level policy must compare candidate domains without access to their complete intra-domain states, whereas the low-level policy requires fine-grained local information to select a physical node. Moreover, the high-level action determines which low-level policy and node action space are activated. The resulting placement quality can be fully evaluated only after the remaining VNF placements and link mappings are completed. Accordingly, the formulation specifies three problem-dependent components: heterogeneous observations for the two levels, conditional domain-to-node actions, and a shared end-to-end embedding reward.

1) *MDP Formulation*: **State**: We represent the physical and virtual states as matrices composed of multiple physical and virtual nodes, respectively. Each row vector within these matrices encodes the resource and topological attributes of a corresponding node, defined as:

$$\begin{aligned} s_d^p &= [\vec{f}_{n_{d,1}^p}, \vec{f}_{n_{d,2}^p}, \dots, \vec{f}_{n_{d,|N^p|}^p}]^T, \\ s^v &= [\vec{f}_{n_1^v}, \vec{f}_{n_2^v}, \dots, \vec{f}_{n_{|N^v|}^v}]^T, \end{aligned} \quad (5)$$

where the feature vector of the virtual node is $\vec{f}_{n_i^v} = [C_{n_i^v}, M_{n_i^v}, \widehat{BW}_{n_i^v}, D_{n_i^v}] \in \mathbb{R}^{F^v \times 1}$, and the feature vector of the physical node is $\vec{f}_{n_{d,k}^p} = [\widehat{C}_{n_{d,k}^p}, \widehat{M}_{n_{d,k}^p}, \widehat{BW}_{n_{d,k}^p}, D_{n_{d,k}^p}, DT, U_{C,d}, U_{M,d}, U_{BW,d}] \in \mathbb{R}^{F^p \times 1}$. Here, F^v and F^p denote the feature dimensions. The terms $\widehat{C}$, $\widehat{M}$, and $\widehat{BW}$ represent the residual resources of a physical node $n_{d,k}^p$, D denotes the node degree, and DT represents the distance between node n_i^p and the previously deployed node. Additionally, $U_{C,d}$, $U_{M,d}$, and $U_{BW,d}$ quantify the resource utilization of network domain d . These quantities are computed as follows:

$$\widehat{C}_{n_{d,k}^p} = C_{n_{d,k}^p} - \sum_{n_i^v \in \mathcal{H}^t(n_{d,k}^p)} C_{n_i^v}. \quad (6)$$

$$\widehat{M}_{n_{d,k}^p} = M_{n_{d,k}^p} - \sum_{n_i^v \in \mathcal{H}^t(n_{d,k}^p)} M_{n_i^v}. \quad (7)$$

$$\widehat{BW}_{n_{d,k}^p} = \frac{\sum_{l_{d,j}^p \in \mathcal{L}(n_{d,k}^p)} (BW_{l_{d,j}^p} - \sum_{l_i^v \in \mathcal{H}(l_{d,j}^p)} BW_{l_i^v})}{D_{n_{d,k}^p}}. \quad (8)$$

$$U_{C,d} = \frac{\sum_{n_{d,k}^p \in N_d^p} \sum_{n_i^v \in \mathcal{H}^t(n_{d,k}^p)} C_{n_i^v}}{\sum_{n_{d,k}^p \in N_d^p} C_{n_{d,k}^p}}. \quad (9)$$

$$U_{M,d} = \frac{\sum_{n_{d,k}^p \in N_d^p} \sum_{n_i^v \in \mathcal{H}^t(n_{d,k}^p)} M_{n_i^v}}{\sum_{n_{d,k}^p \in N_d^p} M_{n_{d,k}^p}}. \quad (10)$$

$$U_{BW,d} = \frac{\sum_{l_{d,k}^p \in L_d^p} \sum_{l_i^v \in \mathcal{H}^t(l_{d,k}^p)} BW_{l_i^v}}{\sum_{l_{d,k}^p \in L_d^p} BW_{l_{d,k}^p}}. \quad (11)$$

where $\mathcal{H}^t(n_{d,k}^p)$ and $\mathcal{H}^t(l_{d,k}^p)$ denote the set of virtual nodes hosted on physical node $n_{d,k}^p$ and the set of virtual links hosted on physical link $l_{d,k}^p$ at time t , respectively. And set $\mathcal{L}(n_{d,k}^p)$ denotes the adjacent edges of node $n_{d,k}^p$. The joint state space of the high-level agent is defined as the collection of network resource states across all domains. Let s_i^p denote the state of domain i , the high-level state is formed by concatenating these domain states: $S^H = [(\mathcal{E}(s_1^p))^\top, (\mathcal{E}(s_2^p))^\top, \dots, (\mathcal{E}(s_d^p))^\top]^\top \in \mathbb{R}^{|N^p| \times F^p}$, where $\mathcal{E}(\cdot)$ represents the encode process. The state space of the low-level agent consists of the state of the selected domain, denoted as s_i^p .

Action: In the multi-agent hierarchical architecture illustrated in Fig. 3, we process VNFs sequentially according to a topological order. For each VNF to be embedded, the decision process is decomposed into a two-level action: (1) a high-level domain-selection agent chooses a target domain from a discrete action set $A^H = \{1, 2, \dots, d\}$; and (2) within the selected domain d , a low-level node-placement agent selects a target physical node from the discrete action set $A^L = \{1, 2, \dots, |N_d^p|\}$. Accordingly, the hierarchical action space can be viewed as a constrained discrete set, the size of which varies dynamically with the number of physical nodes in each domain, bounded by $\sum_{d=1}^D |N_d^p|$.

Reward: To maximize the benefit of SFC embedding, we define the reward based on the $R2C$ obtained from embedding an SFC. The reward function is formulated as:

$$\begin{aligned} R &= \{r_1, r_2, \dots, r_{|N^v|}\} \\ \text{s.t. } r_i &= \begin{cases} (C_{n_i^v} + M_{n_i^v}) + \frac{1}{|L^v|+1} \sum_{l_i^v \in L^v} BW_{l_i^v}, \\ \left[\sum_{n_i^v \in N^v, n_{d,k}^p = \hat{r}_n(n_i^v)} (C_{n_{d,k}^p} u_C + M_{n_{d,k}^p} u_M) \right. \\ \left. + \sum_{l_i^v \in L^v, P^p = \hat{r}_l(l_i^v)} \left(\sum_{l_{d,k}^p \in P^p} BW_{l_{d,k}^p} \right) u_{BW} \right] \\ \text{if } \Omega(g^v) = 1, \\ -\alpha, \text{ otherwise.} \end{cases} \end{aligned} \quad (12)$$

If the SFC embedding fails (i.e., $\Omega(g^v) = 0$), we assign a small negative reward $R = -\alpha$ to penalize decisions that consume embedding steps without yielding a feasible end-to-end solution. It is important to note that this reward is delayed, in contrast to immediate rewards provided after each VNF placement. This design is chosen because the $R2C$ can only be observed after both the problem **P1** and **P2** are solved. We employ a terminal reward because VNF-level immediate rewards may overestimate locally attractive actions. For instance, placing the current VNF on a specific node might appear to reduce node-side costs; however, it could force subsequent links to traverse longer paths and consume excessive bandwidth, thereby increasing the overall cost or causing embedding failure. Conversely, the terminal $R2C$

jointly reflects (i) whether the embedding succeeds end-to-end (affecting the acceptance ratio Acc) and (ii) the overall cost efficiency upon success (affecting $R2C$). This mechanism mitigates the risk of local greedy choices degrading the global SFC objective.

2) *Policy Design*: We deploy agents across the domain network using a hierarchical policy framework defined by π^H (global domain selection) and π^L (local node placement). As illustrated in Fig. 3, the architecture employs two decoupled encoders to process heterogeneous state representations. Specifically, state features are projected into latent spaces via parallel Multilayer Perceptrons (MLPs):

$$h_d^p = s_d^p W^p, \quad h^v = s^v W^v, \quad (13)$$

where $W^p \in \mathbb{R}^{F^p \times H}$ and $W^v \in \mathbb{R}^{F^v \times H}$ are learnable weight matrices. These embeddings are concatenated and fused to produce task-specific logits $\vec{Z}^H$ and $\vec{Z}^L$ for the high-level and low-level policies, respectively:

$$\vec{Z}^H = \sigma \left(\left[(h_1^p)^\top, (h_2^p)^\top, \dots, (h_d^p)^\top \right]^\top \parallel h^v \right) W^H, \quad (14a)$$

$$\vec{Z}^L = \sigma ([h_d^p \parallel h^v]) W^L, \quad (14b)$$

where $W^H \in \mathbb{R}^{2H \times 1}$ and $W^L \in \mathbb{R}^{2H \times 1}$ are learnable weight matrices, and σ denotes the activation function. The global logits $\vec{Z}^H \in \mathbb{R}^{|N^p| \times 1}$ are formed by aggregating the domain-specific scores $\left[(\vec{Z}_1^H)^\top, \dots, (\vec{Z}_d^H)^\top \right]^\top$. Since h_d^p is learned from s_d^p to retain information useful for domain selection, correlations with the underlying resource state may remain. The framework does not employ differential privacy, cryptographic protection, or an information-theoretic non-inference mechanism. Accordingly, W2HP reduces direct exposure of fine-grained intra-domain states but does not assume the exported representation is completely free of information leakage. The high-level policy π^H computes the probability of selecting domain d by aggregating the softmax-normalized scores of its constituent nodes:

$$P_d^H = \sum_{n_{d,i}^p \in N_d^p} \text{Softmax}(\vec{Z}_{n_{d,i}^p}^H). \quad (15)$$

Similarly, the low-level policy π^L generates a categorical distribution over local nodes via a softmax operation on $\vec{Z}^L \in \mathbb{R}^{|N_d^p| \times 1}$, defined as:

$$P_{n_{d,i}^p}^L = \text{Softmax}(\vec{Z}^L + \vec{m}_d) n_{d,i}^p, \quad (16)$$

where $\text{Softmax}(\vec{x}_i) = \frac{e^{\vec{x}_i}}{\sum_j e^{\vec{x}_j}}$, and $\vec{m}_d$ represents the resource availability mask. The mask entries corresponding to node positions with insufficient resources to embed the current VNF are set to $-\infty$.

D. Policy Model Training Methodology

We employ the MAPPO method to train the hierarchical multi-agent network, adhering to the CTDE framework. Algorithm 2 delineates the complete interaction process. Specifically, all agents function as independent PPO agents while sharing a centralized value network V_ϕ for

Algorithm 2 Centralized-Decentralized Training Process

- 1: **Initialize**: high-level policy π^H , low-level policies π^L , centralized value network V_ϕ , and on-policy trajectory buffer $\mathcal{B}$.
- 2: **while** system is running **do**
- 3: Receive an SFC request with VNF set N^v and virtual-link set L^v .
- 4: Initialize $N_{\text{map}} \leftarrow \emptyset$, $L_{\text{map}} \leftarrow \emptyset$, and $\tau_{\text{ep}} \leftarrow \emptyset$, where N_{map} and L_{map} store the VNF-to-node and virtual-link-to-path mappings, respectively, and τ_{ep} stores the trajectory of the current SFC request.
- 5: **for** each VNF $n_i^v \in N^v$ **do**
- 6: Obtain the high-level state S^H .
- 7: Select domain $A^H \sim \pi^H(\cdot | S^H)$.
- 8: Obtain the low-level state S^L conditioned on A^H .
- 9: Select physical node $A^L \sim \pi^L(\cdot | S^L)$.
- 10: **if** A^L satisfies the resource constraints **then**
- 11: $N_{\text{map}} \leftarrow N_{\text{map}} \cup \{(n_i^v, A^L)\}$.
- 12: **end if**
- 13: Store the current transition in τ_{ep} .
- 14: **end for**
- 15: **if** $|N_{\text{map}}| = |N^v|$ **then**
- 16: **for** each virtual link $l_j^v \in L^v$ **do**
- 17: Obtain physical path p_j for l_j^v using CRM.
- 18: **if** p_j is feasible **then**
- 19: $L_{\text{map}} \leftarrow L_{\text{map}} \cup \{(l_j^v, p_j)\}$.
- 20: **end if**
- 21: **end for**
- 22: **end if**
- 23: Calculate the terminal reward R according to the feasibility and quality of N_{map} and L_{map} .
- 24: Assign R to the transitions in τ_{ep} .
- 25: Append τ_{ep} to the on-policy trajectory buffer $\mathcal{B}$.
- 26: **if** the policy-update condition is satisfied **then**
- 27: Update π^H , π^L , and V_ϕ using the trajectories in $\mathcal{B}$ according to Algorithm 3.
- 28: Clear $\mathcal{B}$ after the update.
- 29: **end if**
- 30: **end while**

evaluation. The centralized critic follows the same cross-domain information boundary as the high-level policy. The critic uses the abstract representations reported by the domain controllers together with the SFC state, rather than the raw intra-domain state matrices. Let the critic input be $S^C = [[(h_1^p)^\top, \dots, (h_d^p)^\top] \parallel (h^v)^\top]^\top$, and the value function is evaluated as $V_\phi(S^C)$. Consequently, centralized training does not require domain controllers to disclose their fine-grained state matrices to the central controller. For an SFC arriving at time t , the decision trajectory is defined as $\tau = \{(S_k^H, A_k^H, S_k^L, A_k^L, R_k^H, R_k^L)\}_{k=0}^T$, which encompasses all VNF placement steps.

The objective is to optimize the policy parameters $\theta = \{\theta^H, \theta^L\}$ and the critic parameters ϕ to maximize the expected

cumulative reward:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \gamma^t R_t^{H,L} \right]. \quad (17)$$

During training, we minimize a combined loss function. The policy loss $\mathcal{L}^{\text{CLIP}}$ for both high-level and low-level agents is defined using the standard PPO clipped objective:

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E} \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right], \quad (18)$$

where $r_t(\theta) = \frac{\pi_\theta(A_t^{H,L} | S_t^{H,L})}{\pi_{\theta_{\text{old}}}(A_t^{H,L} | S_t^{H,L})}$ denotes the probability ratio, and ϵ represents the clipping hyperparameter. The term $\hat{A}_t$ is the Generalized Advantage Estimation (GAE), computed as:

$$\hat{A}_t = \sum_{l=0}^{T-t-1} (\gamma \lambda)^l \delta_{t+l}, \delta_t = R_t^{H,L} + \gamma V_\phi(S_{t+1}^{H,L}) - V_\phi(S_t^{H,L}), \quad (19)$$

The discount factor γ controls the contribution of future rewards, whereas λ is the GAE parameter that controls the bias-variance trade-off of the advantage estimate. In all experiments, we set $\gamma = 0.95$ and $\lambda = 0.95$.

The actor and critic are optimized separately. For both the high-level and low-level policies, the actor loss is defined as

$$\mathcal{L}_{\text{actor}}(\theta) = -\mathcal{L}^{\text{CLIP}}(\theta) - \beta_H \mathcal{H}(\pi_\theta), \quad (20)$$

where $\mathcal{H}(\pi_\theta)$ denotes the policy entropy and β_H is the entropy coefficient used to balance exploration and exploitation. The centralized critic minimizes the value function loss $\mathcal{L}^{\text{VF}}$, defined as the mean squared error between the predicted value and the target return:

$$\mathcal{L}_{\text{critic}}(\phi) = \mathcal{L}^{\text{VF}}(\phi) = \mathbb{E} \left[\left(V_\phi(s_t) - \hat{R}_t \right)^2 \right], \quad (21)$$

where $\hat{R}_t = \hat{A}_t + V_\phi(s_t)$ represents the computed return. The parameter update procedure is detailed in Algorithm 3.

E. Complexity Analysis

Let $|N^v|$ denote the number of VNFs in an SFC, D the number of domains, $|N_d^p|$ the number of physical nodes in domain d , and $N_{\max} = \max_d |N_d^p|$. For each VNF, W2HP first selects among D candidate domains and then activates only the selected local agent to choose a physical node within that domain. Thus, placing all VNFs has complexity $\mathcal{O}(|N^v|(D + N_{\max}))$. In contrast, a flat placement policy evaluates all physical nodes, with complexity $\mathcal{O}(|N^v||N^p|)$, where $|N^p| = \sum_{d=1}^D |N_d^p|$. For a roughly balanced multi-domain network, $N_{\max} \approx |N^p|/D$, and thus the complexity of W2HP becomes $\mathcal{O}(|N^v|(D + \frac{|N^p|}{D}))$. This shows that the hierarchical Which-and-Where mechanism reduces action-evaluation overhead by avoiding global node enumeration for each VNF. For link mapping, CRM performs shortest-path search over the abstract topology rather than the complete physical network. Let N_A and M_A denote the numbers of nodes and links in the abstract topology, respectively. For each inter-domain virtual link, the source and destination domains compute access costs

Algorithm 3 Iterative Update with Accumulated Trajectories

- 1: **Input:** Batch of trajectories $\mathcal{B}$, Batch size B .
- 2: **Parameters:** High-level θ^H , Low-level $\{\theta^{L,d}\}$, Critic ϕ .
- 3: Compute target values and advantages for all transitions in $\mathcal{B}$ using GAE.
- 4: **for** epoch $k = 1$ to K_{epochs} **do**
- 5: Shuffle trajectories and divide into mini-batches of size M .
- 6: **for** each mini-batch **do**
- 7: // Update High-level Policy
- 8: Compute $\mathcal{L}^{\text{CLIP}}(\theta^H)$ using high-level transitions.
- 9: $\theta^H \leftarrow \theta^H - \eta \nabla_{\theta^H} \mathcal{L}_{\text{total}}(\theta^H)$.
- 10: // Update Low-level Policies
- 11: **for** each domain d **do**
- 12: Compute $\mathcal{L}^{\text{CLIP}}(\theta^{L,d})$ using domain-specific transitions.
- 13: $\theta^{L,d} \leftarrow \theta^{L,d} - \eta \nabla_{\theta^{L,d}} \mathcal{L}_{\text{total}}(\theta^{L,d})$.
- 14: **end for**
- 15: // Update Centralized Critic
- 16: Compute $\mathcal{L}^{\text{VF}}(\phi)$.
- 17: $\phi \leftarrow \phi - \eta \nabla_\phi \mathcal{L}^{\text{VF}}(\phi)$.
- 18: **end for**
- 19: **end for**
- 20: Clear buffer $\mathcal{B}$.

from the mapped endpoints to their boundary nodes, followed by a shortest-path search over the abstract topology. Thus, the worst-case complexity of one CRM query is $\mathcal{O}(L_{\max} + N_{\max} \log N_{\max} + M_A + N_A \log N_A)$, where $N_{\max} = \max_d |N_d^p|$ and $L_{\max} = \max_d |L_d^p|$. Hence, routing all $|L^v|$ virtual links requires $\mathcal{O}(|L^v|(L_{\max} + N_{\max} \log N_{\max} + M_A + N_A \log N_A))$.

F. Communication Overhead Analysis

We further characterize the control-plane communication between domain controllers and the central controller. Let b denote the number of bits representing one numeric state value. For domain d , directly reporting its fine-grained physical state $\mathbf{s}_d^p \in \mathbb{R}^{|N_d^p| \times F_p}$ requires $b|N_d^p|F_p$ bits per state update. In W2HP, the domain controller performs high-level encoding locally and reports only $\mathbf{z}_d^H \in \mathbb{R}^{|N_d^p| \times 1}$, requiring $b|N_d^p|$ bits. Therefore, the payload ratio of abstract-state reporting to raw-state reporting is

$$\rho_{\text{state}} = \frac{\sum_{d=1}^D |N_d^p|}{F_p \sum_{d=1}^D |N_d^p|} = \frac{1}{F_p}. \quad (22)$$

For the physical-state representation used in this work, $F_p = 8$, so the reported state payload is 12.5% of that for transmitting the complete physical-state matrices, corresponding to an 87.5% reduction in transmitted numeric values. If the high-level state is refreshed for every VNF placement decision, an SFC with $|N^v|$ VNFs requires $b|N^v||N^p|$ bits for domain-state reports, compared with $b|N^v||N^p|F_p$ bits under raw-state reporting. CRM introduces a separate routing signaling cost. Let B_d denote the number of border nodes in domain d . Maintaining the full-mesh topology abstraction requires $\binom{B_d}{2}$

intra-domain border-pair costs from domain d whenever the abstraction is refreshed. Thus, a complete abstraction update contains $\sum_{d=1}^D \binom{B_d}{2}$ numeric cost values. For an inter-domain routing query with mapped endpoints in domains d_s and d_t , the query-specific access information contains $B_{d_s} + B_{d_t}$ endpoint-to-border cost values. These quantities characterize the protocol-independent signaling payload; protocol headers, serialization overhead, and controller-to-controller transmission latency depend on the specific SDN implementation and are not included.

IV. IMPLEMENTATION AND PERFORMANCE ANALYSIS

A. Multi-Domain Network

We construct a multi-domain network simulation environment based on Python, utilizing the Waxman random graph model [33] to generate the internal topology of each domain. Given N^{pd} nodes in domain d , the Waxman model determines whether a link exists between nodes i and j through the probability $P(i, j) = \alpha \cdot \exp\left(-\frac{d_{ij}}{\beta \cdot L}\right)$, where $\alpha \in (0, 1]$ controls the edge density, $\beta \in (0, 1]$ governs the distance decay rate, d_{ij} denotes the Euclidean distance between nodes, and L represents the maximum possible distance within the domain. The Waxman model has been widely adopted in routing and network simulation studies [13], [34], [35]. The multi-domain network topologies used for simulation is given as: **(1) D3N20**: A topology comprising 3 network domains, each containing 20 physical nodes. **(2) D4N20**: A topology comprising 4 network domains, each containing 20 physical nodes. **(3) D10NX**: A topology comprising 10 network domains, each containing $X \in [20, 30]$ physical nodes. **(4) Cogentco**: A real-world IP backbone topology obtained from the Internet Topology Zoo [36], which was reconstructed from network maps publicly released by network operators.

The resource attributes of physical and virtual nodes and links are detailed in Table III, following the resource allocation scheme presented in [11], [13].

TABLE III: Simulation network parameter settings

Physical Network		
Physical Node	CPU (unit)	$\mathcal{U}[100, 600]$
	Memory (MB)	$\mathcal{U}[100, 600]$
Physical Link	Bandwidth (Mbps)	$\mathcal{U}[500, 2000]$
Service Function Chain		
Length		$\mathcal{U}_{\text{int}}[2, 10]$
Virtual Node	CPU (unit)	$\mathcal{U}[10, 60]$
	Memory (MB)	$\mathcal{U}[10, 60]$
Virtual Link	Bandwidth (Mbps)	$\mathcal{U}[50, 150]$

B. Convergence Analysis

Fig. 4 reports cumulative training reward on four network topologies, including three synthetic multi-domain topologies and the real-world Cogentco topology. In addition to W2HP, we include four variants to examine the effects of encoder design and high-level domain selection: W2HP with a shared encoder, W2HP with IPPO-based domain selection, W2HP with random domain selection, and W2HP with greedy domain

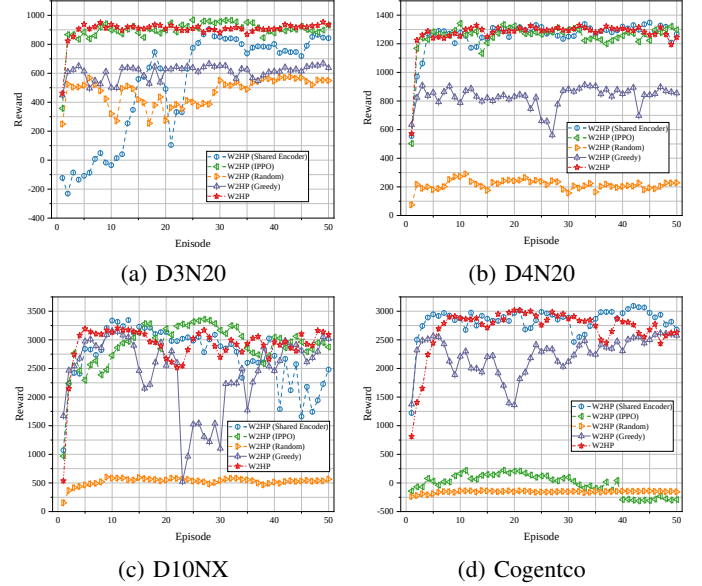


Fig. 4: Training rewards of W2HP and its variants under different network topologies.

selection. As shown in Fig. 4, W2HP reaches a stable high-reward region within few training episodes on D3N20 and D4N20, indicating that the hierarchical policy can be effectively optimized in small-scale multi-domain environments. On the larger D10NX topology, the reward curves show more visible fluctuations because the enlarged action space and tighter SFC embedding constraints make placement and routing decisions more coupled. Nevertheless, W2HP maintains a consistently high reward level after the initial training stage.

The comparison among variants further illustrates the role of the proposed design. The shared-encoder variant learns effective policies in several cases, but its reward is less stable on complex topologies, suggesting that separating high- and low-level representations helps reduce interference between domain selection and node placement. The IPPO, random, and greedy variants also show that replacing learned hierarchical coordination with less structured domain-selection strategies can lead to unstable or lower reward, especially on the real-world Cogentco topology.

C. Ablation on the Which Decision

To isolate the effect of high-level domain selection, we compare W2HP with two variants retaining the same intra-domain placement and routing mechanisms but replacing the learned Which policy. W2HP (Random) randomly selects a candidate domain, whereas W2HP (Greedy) selects the domain using the available-resource heuristic. All three variants use the same network settings, request workload, and training configuration. Table IV reports their average performance over the last ten training episodes at an arrival rate of 0.6 to characterize stabilized behavior.

Random domain assignment substantially degrades placement performance across all four environments. Compared with W2HP (Random), W2HP improves the acceptance ratio by 9.07, 27.78, 42.17, and 66.91 percentage points (pp) on

TABLE IV: Ablation results of different *Which* strategies.

Topology	Method	Acc	LT_R2C	Reward
D3N20	W2HP	0.4862	1231	928.41
	W2HP (Random)	0.3955	888	557.22
	W2HP (Greedy)	0.3900	986	637.49
D4N20	W2HP	0.5720	1482	1273.81
	W2HP (Random)	0.2942	593	208.93
	W2HP (Greedy)	0.4673	1191	848.42
D10NX	W2HP	0.9293	2620	2987.27
	W2HP (Random)	0.5076	774	533.85
	W2HP (Greedy)	0.8880	2538	2841.16
Cogentco	W2HP	0.8645	2411	2646.01
	W2HP (Random)	0.1954	240	-146.33
	W2HP (Greedy)	0.8600	2358	2554.54

D3N20, D4N20, D10NX, and Cogentco, respectively. The corresponding *LT_R2C* improvements are 343, 889, 1846, and 2171. This pronounced degradation under random selection confirms that the subsequent Where policy cannot reliably compensate for an inappropriate domain assignment once its node-selection space is restricted to the selected domain. W2HP (Greedy) provides a stronger comparison by incorporating available-resource information into domain selection. Nevertheless, W2HP achieves consistently higher stabilized performance. Compared with W2HP (Greedy), W2HP improves the acceptance ratio by 9.62 and 10.47 pp on D3N20 and D4N20, respectively, while increasing *LT_R2C* by 245 and 291. Gains are smaller on D10NX and Cogentco, where the greedy heuristic already provides competitive domain assignments: the acceptance-ratio improvements are 4.13 and 0.45 pp, and the *LT_R2C* improvements are 81 and 53, respectively. These results show that the domain-selection strategy materially affects subsequent placement performance and quantitatively support the importance of the Which decision. They also show that the additional benefit of learning the Which policy over a resource-based heuristic depends on the topology and resource distribution.

D. Comparative Analysis

1) *Baselines*: We conduct a comprehensive comparison of the proposed W2HP against three algorithms: GCN-VNE [37], HFL-VNE [13], and NRM [23], in terms of acceptance rate (*Acc*), average end-to-end delay (Delay), long-term revenue-to-cost ratio (*LT_R2C*), and unit revenue-to-cost ratio (*UR2C*). We first evaluate all four algorithms under varying request arrival rates on three synthetic multi-domain topologies, i.e., D3N20, D4N20, and D10NX, as well as the real-world Cogentco backbone topology. We then investigate the impact of SFC length on placement performance.

2) *Performance Under Different Arrival Rates*: In this experiment, the request arrival rate increases from 0.1 to 1.0 with a step size of 0.1. Each test interval runs for 5,000 time units to ensure sufficient data collection. Fig. 5 reports the *Acc* and *R2C* performance of the four algorithms on D3N20, D4N20, D10NX and Cogentco.

Analysis of Acc: As shown in Figs. 5(a)-(d), the acceptance rate generally decreases as request arrival rate increases be-

cause physical resources become increasingly constrained under heavier traffic loads. W2HP consistently maintains a higher *Acc* than the baselines across all four environments. Averaged over all arrival rates, W2HP achieves *Acc* values of 58.22%, 66.21%, 98.55%, and 92.69% on D3N20, D4N20, D10NX, and Cogentco, respectively. On D3N20 and D4N20, W2HP improves average *Acc* over GCN-VNE by 9.38% and 8.97%, respectively, and over HFL-VNE by 8.10% and 8.76%. On the larger D10NX topology, W2HP achieves an average *Acc* of 0.9855, exceeding GCN-VNE and HFL-VNE by 5.30% and 2.55%, respectively. More importantly, this advantage is preserved on the real-world Cogentco, where W2HP achieves an average *Acc* of 0.9269, compared with 80.91% for GCN-VNE and 81.97% for HFL-VNE. This corresponds to improvements of 11.78% and 10.72%, respectively. These results indicate that explicitly coordinating domain selection and intra-domain placement remains effective across different synthetic network scales and under a realistic backbone topology.

Analysis of Delay: The delay of an SFC includes accumulated communication delay along its physical routing path and the processing delay of its deployed VNFs. Figs. 5(e)-(h) report average end-to-end delay of successfully embedded SFCs. On the three synthetic environments, W2HP maintains competitive or lower delay while admitting more requests. Its average Delay values are 314.76, 322.34, and 296.09 on D3N20, D4N20, and D10NX, respectively. Compared with GCN-VNE, these correspond to average delay reductions of 3.60%, 4.97%, and 10.82%. The reduction becomes more evident on D10NX, where the larger network provides more candidate domain and node combinations, giving W2HP greater flexibility to identify placement decisions with shorter routing paths. On the Cogentco topology, W2HP achieves an average Delay of 339.86, which is 4.50% lower than GCN-VNE's 355.88 but higher than HFL-VNE's 324.31. This result should be interpreted with the acceptance rate: HFL-VNE accepts only 0.8197 of the requests on average, whereas W2HP accepts 0.9269. Since Delay is averaged over successfully embedded requests, a lower Delay alone does not necessarily indicate better placement performance, as rejecting more resource-constrained requests can reduce the average delay of the remaining accepted requests. W2HP achieves a more favorable balance between request admission and latency. NRM exhibits substantially higher delay in all four environments, indicating that resource-ranking-based placement cannot effectively control the routing cost induced by sequential VNF placement.

Analysis of R2C: Figs. 5(i)-(l) show that W2HP achieves the highest *LT_R2C* across all four environments over the evaluated arrival-rate range. Averaged over all arrival rates, W2HP obtains *LT_R2C* values of 1105, 1331, 2579, and 2225 on D3N20, D4N20, D10NX, and Cogentco, respectively. Compared with HFL-VNE, the strongest learning-based baseline for *LT_R2C*, W2HP improves the corresponding values by 15.61%, 19.00%, 7.12%, and 10.86%, respectively. The consistent advantage on Cogentco further indicates that the performance gain does not depend on the structural characteristics of Waxman-generated synthetic topologies. By coordinating domain-level resource availability with intra-domain node placement, W2HP admits more SFC requests

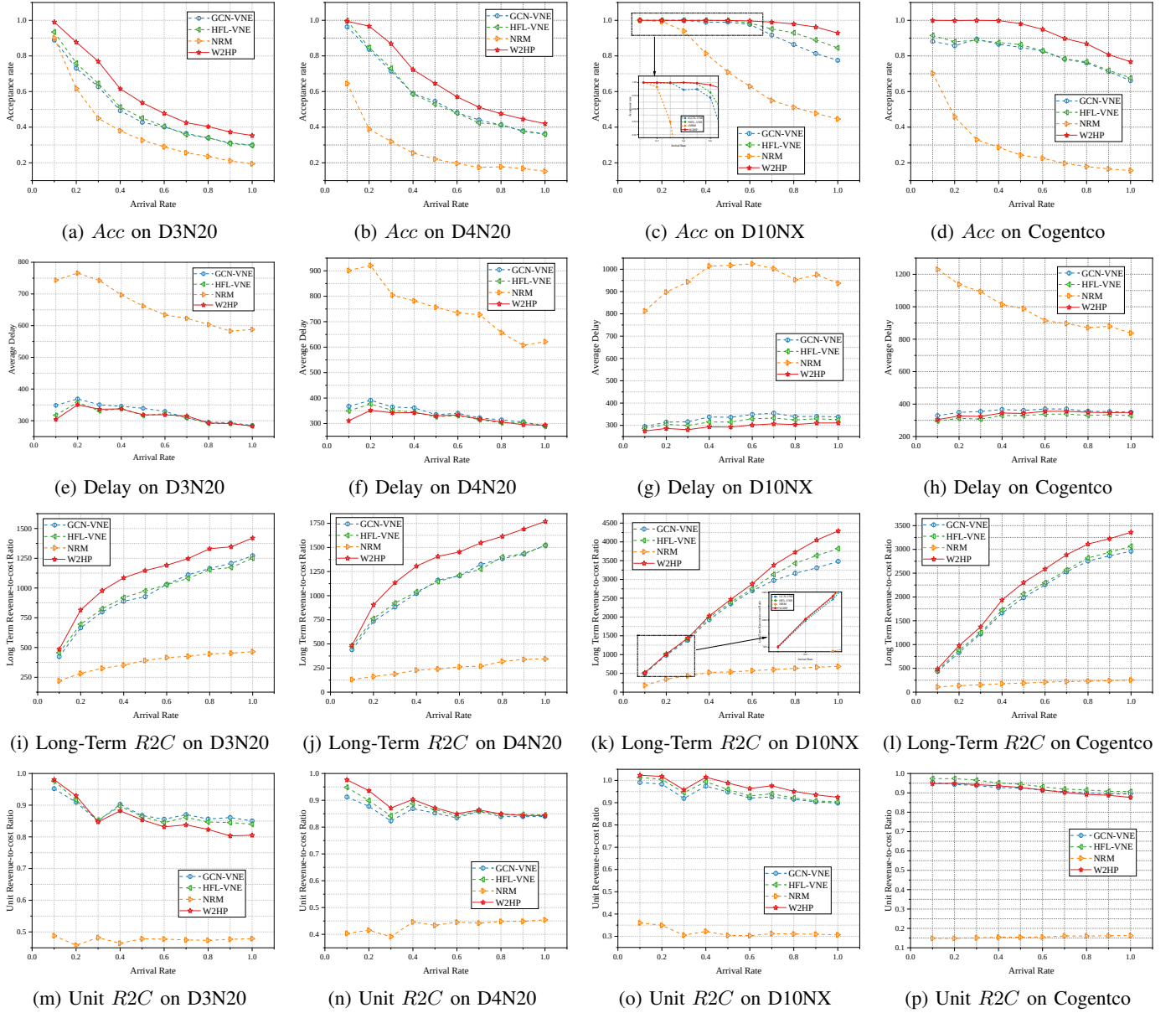


Fig. 5: Performance comparison in terms of acceptance ratio, delay, long-term $R2C$, and unit $R2C$ under different network.

while controlling embedding costs, thereby improving long-term system utility.

Analysis of Unit $R2C$: Figs. 5(m)-(p) report the average revenue-to-cost ratio of each successfully embedded SFC. Unlike LT_R2C , $UR2C$ measures the embedding efficiency of an individual accepted request and thus does not directly reflect request admission. On D3N20, W2HP obtains an average $UR2C$ of 0.8594, slightly below GCN-VNE and HFL-VNE, while achieving substantially higher Acc and LT_R2C . As network scale increases, this trade-off becomes less pronounced: W2HP achieves the highest average $UR2C$ on both D4N20 and D10NX, reaching 0.8808 and 0.9752, respectively. A similar trade-off is observed on Cogentco. W2HP obtains an average $UR2C$ of 0.9176, which is 2.27% lower than HFL-VNE's 0.9389. However, W2HP improves the average Acc from 0.8197 to 0.9269 and the LT_R2C from 0.4014

to 0.4450. Therefore, W2HP does not simply optimize the efficiency of individually accepted requests; it seeks a better system-level balance among request admission, placement cost, and latency. The consistently higher LT_R2C shows that this trade-off yields greater accumulated utility over the evaluation period.

3) Impact of SFC Length: Considering that long-path SFCs are more likely to be deployed across different network domains, it is necessary to evaluate the impact of SFC length on embedding quality. We generate SFC requests with fixed lengths of 5, 10, and 15 VNFs, with the test results presented in Tab. V.

Analysis of Acc: SFCs containing fewer VNFs are more easily embedded into the network. Conversely, as the number of VNFs increases, the functional workflow of the SFC becomes more complex, resulting in higher failure rates. W2HP

TABLE V: Performance under different SFC lengths

Method	D3N20						D4N20						D10N20					
	5		10		15		5		10		15		5		10		15	
	Acc	R2C	Acc	R2C	Acc	R2C	Acc	R2C	Acc	R2C	Acc	R2C	Acc	R2C	Acc	R2C	Acc	R2C
GCN-VNE	0.433	915.52	0.199	425.75	0.118	253.48	0.525	1096.71	0.209	411.18	0.113	205.83	0.980	3243.58	0.592	1770.45	0.372	1088.36
HFL-VNE	0.447	941.59	0.197	423.88	0.119	253.57	0.521	1099.60	0.211	431.18	0.114	217.90	0.992	3325.24	0.650	1969.84	0.399	1186.39
NRM	0.299	338.76	0.163	209.26	0.116	155.94	0.189	186.98	0.105	114.05	0.075	85.42	0.584	606.34	0.311	322.00	0.230	259.59
W2HP	0.566	1159.22	0.263	563.56	0.163	351.96	0.696	1483.36	0.334	724.56	0.210	455.79	1.000	3457.75	0.793	2492.82	0.550	1703.18

achieves the best performance across all three test configurations, demonstrating its capability to effectively handle SFC embedding requirements of varying complexity levels.

Analysis of R2C: More complex embedded SFCs typically consume greater amounts of physical resources. This constraint causes GCN-VNE and HFL-VNE to preferentially embed simple resource requests while rejecting an excessive number of complex requests. In contrast, W2HP, leveraging its hierarchical architecture design, determines the optimal VNF placement based on the resource state of each domain. By partitioning long chains into manageable segments across network domains, W2HP significantly mitigates the agent's tendency to reject complex requests.

V. CONCLUSION

In this paper, we addressed the MD-SFCP problem by proposing W2HP, a hierarchical MARL framework that explicitly models both domain-selection and intra-domain placement decisions. Unlike existing schemes that mainly optimize intra-domain placement under an implicit domain-selection assumption, W2HP introduces a two-tier architecture that first selects the hosting domain and then performs fine-grained VNF placement inside the selected domain. The proposed framework makes three key contributions: (1) Firstly, W2HP provides a hierarchical *Which-and-Where* placement model that decomposes MD-SFCP into coordinated classification and deployment tasks, thereby capturing domain-level capacity heterogeneity before node-level decisions; (2) Secondly, W2HP adopts a dual encoder architecture, where high-level encoders generate compact quality scores of resource for domain selection and low-level encoders produce detailed placement policies and these agents are trained cooperatively under a MAPPO-based CTDE paradigm; (3) Thirdly, W2HP preserves topology and resource privacy by allowing domain controllers to report only abstract score vectors to the central controller instead of exposing fine-grained internal states.

Future work will focus on extending W2HP to support online SFC migration and reconfiguration under time-varying traffic and resource dynamics, and on validating the framework on realistic multi-domain testbeds to further assess its deployment overhead and robustness.

REFERENCES

- [1] Y. Wang, C. Yang, T. Li, Y. Ouyang, X. Mi, and Y. Song, "A survey on intent-driven end-to-end 6g mobile communication system," *IEEE Commun. Surveys Tuts.*, pp. 1–1, 2025.
- [2] M. Asgarian, K. Jamshidi, and A. Bohlooli, "An efficient approximation algorithm for service function chaining placement in edge-cloud computing industrial internet of things," *IEEE Internet Things J.*, vol. 11, no. 7, pp. 12 815–12 822, 2024.
- [3] S. Guo, Y. Qi, Y. Jin, W. Li, X. Qiu, and L. Meng, "Endogenous trusted drl-based service function chain orchestration for iot," *IEEE Trans. Comput.*, vol. 71, no. 2, pp. 397–406, 2022.
- [4] J. Cai, Z. Zhou, Z. Huang, W. Dai, and F. R. Yu, "Privacy-preserving deployment mechanism for service function chains across multiple domains," *IEEE Trans. Netw. Service Manag.*, vol. 21, no. 1, pp. 1241–1256, 2024.
- [5] X. Shang, Z. Liu, and Y. Yang, "Online service function chain placement for cost-effectiveness and network congestion control," *IEEE Trans. Comput.*, vol. 71, no. 1, pp. 27–39, 2022.
- [6] D. Zheng and X. Cao, "Provably efficient service function chain embedding and protection in edge networks," *IEEE/ACM Trans. Netw.*, vol. 33, no. 1, pp. 178–193, 2025.
- [7] R. Lin, S. Yu, S. Luo, X. Zhang, J. Wang, and M. Zukerman, "Column generation based service function chaining embedding in multi-domain networks," *IEEE Trans. on Cloud Comput.*, vol. 11, no. 1, pp. 185–199, 2023.
- [8] G. Sun, Y. Li, D. Liao, and V. Chang, "Service function chain orchestration across multiple domains: A full mesh aggregation approach," *IEEE Trans. Netw. Service Manag.*, vol. 15, no. 3, pp. 1175–1191, 2018.
- [9] D. Zhao, Y. Lu, X. Li, Z. Li, and Y. Liu, "Cross-domain service function chain routing: Multiagent reinforcement learning approaches," *IEEE Trans. Circuits Syst. II*, vol. 69, no. 12, pp. 4754–4758, 2022.
- [10] K. Wang, P. Yuan, M. A. Jan, F. Khan, T. R. Gadekallu, S. Kumari, H. Pan, and L. Liu, "Digital twin-assisted service function chaining in multi-domain computing power networks with multi-agent reinforcement learning," *Future Generation Computer Systems*, vol. 158, pp. 294–307, 2024.
- [11] P. Zhang, X. Pang, Y. Bi, H. Yao, H. Pan, and N. Kumar, "Dscd: Delay sensitive cross-domain virtual network embedding algorithm," *IEEE Trans. Netw. Sci. Eng.*, vol. 7, no. 4, pp. 2913–2925, 2020.
- [12] S. Zhang, B. Wang, Q. Huang, Y. Wu, R. Xu, K. Wang, and X. Zhang, "Hybrid service function chains dynamic deployment in multi-domain edge cloud networks," in *2022 IEEE 24th Int Conf on High Performance Computing & Communications; 8th Int Conf on Data Science & Systems; 20th Int Conf on Smart City; 8th Int Conf on Dependability in Sensor, Cloud & Big Data Systems & Application (HPCC/DSS/SmartCity/DependSys)*, 2022, pp. 627–634.
- [13] P. Zhang, N. Chen, S. Li, K.-K. R. Choo, C. Jiang, and S. Wu, "Multi-domain virtual network embedding algorithm based on horizontal federated learning," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 3363–3375, 2023.
- [14] D. Thakur and M. Khatua, "Multi-domain virtual network embedding with dynamic flow migration in software-defined networks," *Journal of Network and Computer Applications*, vol. 162, p. 102639, 2020.
- [15] N. Tourni, O. Bernier, D.-E. Meddour, and A. Ksentini, "On cross-domain service function chain orchestration: An architectural framework," *Computer Networks*, vol. 187, p. 107806, 2021.
- [16] N. Tourni, M. Bagaa, and A. Ksentini, "Hierarchical multi-agent deep reinforcement learning for sfc placement on multiple domains," in *2021 IEEE 46th Conference on Local Computer Networks (LCN)*, 2021, pp. 299–304.
- [17] Y. Zhang, R. Wang, J. Hao, Q. Wu, Y. Teng, P. Wang, and D. Niyato, "Service function chain deployment with vnf-dependent software migration in multi-domain networks," *IEEE Trans. Mobile Comput.*, vol. 24, no. 5, pp. 3685–3702, 2025.
- [18] W. Bao, D. Yuan, B. B. Zhou, and A. Y. Zomaya, "Prune and plant: Efficient placement and parallelism of virtual network functions," *IEEE Trans. Comput.*, vol. 69, no. 6, pp. 800–811, 2020.
- [19] Y. Mao, X. Shang, Y. Liu, and Y. Yang, "Joint virtual network function placement and flow routing in edge-cloud continuum," *IEEE Trans. Comput.*, vol. 73, no. 3, pp. 872–886, 2024.
- [20] Y. Liu, Y. He, Y. Li, X. Wang, K. Zhang, M. Ju, Z. Ma, and Q. Tian, "Cost-oriented and delay-constrained anycasting for service function chain provisioning leveraging cloud-edge collaboration in

space-air-ground integrated networks,” *IEEE Internet Things J.*, vol. 12, no. 4, pp. 4475–4487, 2025.

- [21] Y. Mao, X. Shang, and Y. Yang, “Ant colony based online learning algorithm for service function chain deployment,” in *IEEE INFOCOM 2023 - IEEE Conference on Computer Communications*, 2023, pp. 1–10.
- [22] D. Harutyunyan, N. Shahriar, R. Boutaba, and R. Riggio, “Latency and mobility-aware service function chain placement in 5g networks,” *IEEE Trans. Mobile Comput.*, vol. 21, no. 5, pp. 1697–1709, 2022.
- [23] P. Zhang, H. Yao, and Y. Liu, “Virtual network embedding based on computing, network, and storage resource constraints,” *IEEE Internet Things J.*, vol. 5, no. 5, pp. 3298–3304, 2018.
- [24] Y. Bi, C. C. Meixner, M. Bunyakitanon, X. Vasilakos, R. Nejabati, and D. Simeonidou, “Multi-objective deep reinforcement learning assisted service function chains placement,” *IEEE Trans. Netw. Service Manag.*, vol. 18, no. 4, pp. 4134–4150, 2021.
- [25] J. Zhang, H. Yu, G. Fan, Q. Tang, Z. Li, and J. Xu, “Cold-start-aware cloud-native parallel service function chain caching in edge-cloud network,” *IEEE Internet Things J.*, vol. 11, no. 11, pp. 20340–20356, 2024.
- [26] M. Karimzadeh-Farshbafan, V. Shah-Mansouri, and D. Niyato, “A dynamic reliability-aware service placement for network function virtualization (nfv),” *IEEE J. Sel. Areas Commun.*, vol. 38, no. 2, pp. 318–333, 2020.
- [27] J. Liang, W. Li, and V. C. M. Leung, “Stateful virtual network function decomposition and deployment with reliability guarantee in edge networks,” *IEEE Trans. Netw. Service Manag.*, pp. 1–1, 2025.
- [28] H. Chen, S. Wang, G. Li, L. Nie, X. Wang, and Z. Ning, “Distributed orchestration of service function chains for edge intelligence in the industrial internet of things,” *IEEE Trans. Ind. Informat.*, vol. 18, no. 9, pp. 6244–6254, 2022.
- [29] S. Yang, F. Li, Z. Zhou, X. Chen, Y. Wang, and X. Fu, “Online control of service function chainings across geo-distributed datacenters,” *IEEE Trans. Mobile Comput.*, vol. 22, no. 6, pp. 3558–3571, 2023.
- [30] R. He, B. Ren, J. Xie, D. Guo, and L. Zhao, “Efficient online scheduling of service function chains across multiple geo-distributed regions,” *IEEE Trans. Netw. Service Manag.*, vol. 21, no. 3, pp. 3440–3453, 2024.
- [31] M. Zhang, Y. Ai, and H. Li, “Dpsfc: A distributed delay-aware approach for improving parallelized service function chains placement,” in *2024 IEEE International Symposium on Parallel and Distributed Processing with Applications (ISPA)*, 2024, pp. 1903–1910.
- [32] Y. Huang, T. Yao, Z. Lin, X. Shang, Y. Yuan, L. Cui, and Y. Yang, “Efficient service function chain placement over heterogeneous devices in deviceless edge computing environments,” *IEEE Trans. Comput.*, vol. 74, no. 1, pp. 222–236, 2025.
- [33] B. Waxman, “Routing of multipoint connections,” *IEEE J. Sel. Areas Commun.*, vol. 6, no. 9, pp. 1617–1622, 1988.
- [34] J. Li, R. Wang, and K. Wang, “Service function chaining in industrial internet of things with edge intelligence: A natural actor-critic approach,” *IEEE Trans. Ind. Informat.*, vol. 19, no. 1, pp. 491–502, 2023.
- [35] P. Zhang, Z. Li, M. Guizani, N. Kumar, K. Yu, and J. Wang, “Energy aware space-air-ground integrated network resource orchestration algorithm,” *IEEE Trans. Veh. Technol.*, vol. 73, no. 12, pp. 18950–18960, 2024.
- [36] S. Knight, H. X. Nguyen, N. Falkner, R. Bowden, and M. Roughan, “The internet topology zoo,” *IEEE Journal on Selected Areas in Communications*, vol. 29, no. 9, pp. 1765–1775, 2011.
- [37] P. Zhang, C. Wang, N. Kumar, W. Zhang, and L. Liu, “Dynamic virtual network embedding algorithm based on graph convolution neural network and reinforcement learning,” *IEEE Internet Things J.*, vol. 9, no. 12, pp. 9389–9398, 2022.



Peiyang Zhang is currently a Professor with the College of Computer Science and Technology, China University of Petroleum (East China). He received his Ph.D. in the School of Information and Communication Engineering at University of Beijing University of Posts and Telecommunications in 2019. He has published multiple high quality papers since 2016, such as IEEE T-IFS, IEEE TSC, IEEE T-ITS, IEEE TMC and etc. He served as the Technical Program Committee of IJCAI’25, AAAI’25, AAAI’24, AAAI’23.



Shengpeng Chen is currently working toward the graduate degree with the College of Computer Technology, China University of Petroleum (East China). His research interests include network virtualization and artificial intelligence for networking.



Chunming Rong received the Ph.D. degree from the University of Bergen, Bergen, Norway, in 1998. He works as the Head of the Center for IP-Based Service Innovation, University of Stavanger, Stavanger, Norway. He is also the Co-Founder of two startups bitYoga and Dataunitor in Norway, both received EU Seal of Excellence Award in 2018. He was an Adjunct Senior Scientist leading Big-Data Initiative with NORCE, Oslo, Norway, from 2016 to 2019, the Vice President of CSA Norway Chapter from 2016 to 2017.



Ching-Hsien Hsu is a professor in the Department of Computer Science and Information Engineering, Asia University, and Department of Medical Research, China Medical University Hospital, China Medical University, Taiwan. His research includes high-performance computing, cloud computing, parallel and distributed systems, big data analytics, and ubiquitous/pervasive computing and intelligence. He has published 200 papers in refereed journals, conference proceedings, and book chapters in these areas.



Neeraj Kumar is currently working as a Full Professor with the Department of Computer Science and Engineering, Thapar Institute of Engineering and Technology, Patiala, India. He has published more than 400 technical research papers, which are cited more than 43637 times by researchers across the globe with a current H-index of 114. His broad research areas are green computing and network management, IoT, big data analytics, deep learning, and cyber-security.



Mohsen Guizani (Fellow, IEEE) received the B.S. degree (with Distinction), M.S., and Ph.D. degrees in electrical and computer engineering from Syracuse University, Syracuse, NY, USA. He is currently a Professor of Machine Learning with the Mohamed Bin Zayed University of Artificial Intelligence, Abu Dhabi, UAE. Previously, he worked with different institutions in USA. He is the author of 11 books, more than 1000 publications and several US patents. His research interests include applied machine learning, artificial intelligence, smart city, Internet of Things, intelligent autonomous systems, and cybersecurity.

Things, intelligent autonomous systems, and cybersecurity.